

BM25 Index

Best Match 25 — Theory, Implementation & RAG Integration

[rank_bm25](#) · [Keyword Retrieval](#) · [Hybrid Search](#) · [Document Chunks](#)

Created: July 04, 2026 | 22:42

Table of Contents

1. What is BM25?	3
2. The BM25 Formula — Deep Dive	3
3. Key Parameters: k1 and b	4
4. BM25 Variants in rank_bm25	5
5. BM25 vs Vector Search — When to Use Which	5
6. Installation: rank_bm25	6
7. Building a BM25 Index over Document Chunks	6
8. Querying the BM25 Index	7
9. Full End-to-End Example with Your Document Setup	8
10. Hybrid Retrieval: BM25 + Pinecone	9
11. Reciprocal Rank Fusion (RRF)	10
12. Pros, Cons & Limitations of BM25	11
13. References	12

1. What is BM25?

BM25, short for **Best Match 25**, is a probabilistic ranking function used by search engines to rank documents by relevance to a given query. It was developed by Stephen Robertson and colleagues at City University London in the 1990s and presented at the TREC (Text REtrieval Conference). The '25' refers to the iteration number of the probabilistic model that led to this formulation.

BM25 belongs to the family of **bag-of-words** retrieval models — it looks at which query words appear in a document and how often, but does *not* consider word order or meaning. Despite its simplicity, BM25 remains one of the strongest baselines in information retrieval and is still the backbone of systems like Elasticsearch and Apache Lucene.

★ **Key Concept** In RAG (Retrieval-Augmented Generation) pipelines, BM25 is used as a fast, cheap, keyword-based first stage that complements dense vector search — together they form Hybrid Retrieval, the current gold standard.

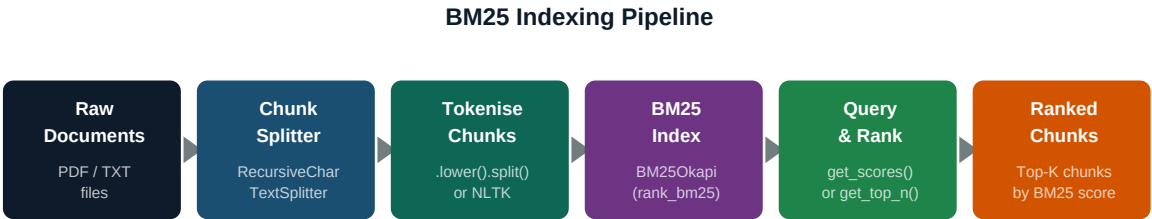


Figure 1 — BM25 pipeline: raw docs → chunks → tokenise → BM25 index → query → ranked results

2. The BM25 Formula — Deep Dive

The BM25 score for a document D given query Q (consisting of terms $q_1, q_2, \dots, q_n$) is:

Sum is over all query terms q_i

BM25 Okapi Scoring Formula

$$\text{Score}(D, Q) = \text{SUM} \frac{\text{IDF}(q_i) \times f(q_i, D) \times (k_1 + 1)}{f(q_i, D) + k_1 \times (1 - b + b \times |D| / \text{avgdl})}$$

IDF(q_i)

Inverse Document Frequency

Penalises terms that appear in many documents

$f(q_i, D)$

Term Frequency in Document D

How often the query term appears

k_1

Saturation Parameter

Controls TF saturation (default: 1.5)

b

Length Normalisation

Adjusts for doc length vs average (default: 0.75)

$|D| / \text{avgdl}$

Length Ratio

Doc length over average document length

Figure 2 — Visual breakdown of each component of the BM25 scoring formula

2.1 IDF — Inverse Document Frequency

IDF penalises terms that appear in many documents (they carry little information). The BM25 IDF formula is:

N = total docs, $n(q_i)$ = docs containing

IDF Score: Common Words Get Low Score, Rare Words Get High Score

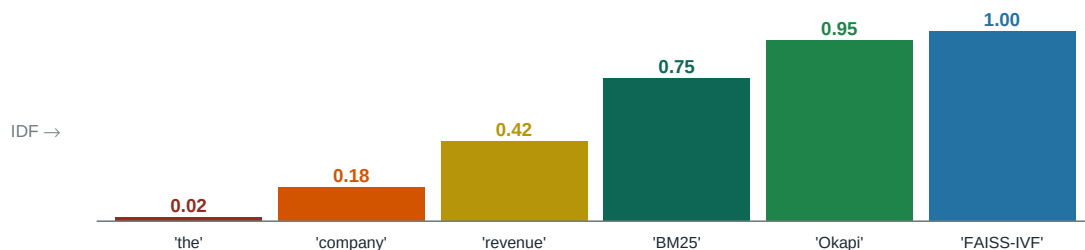
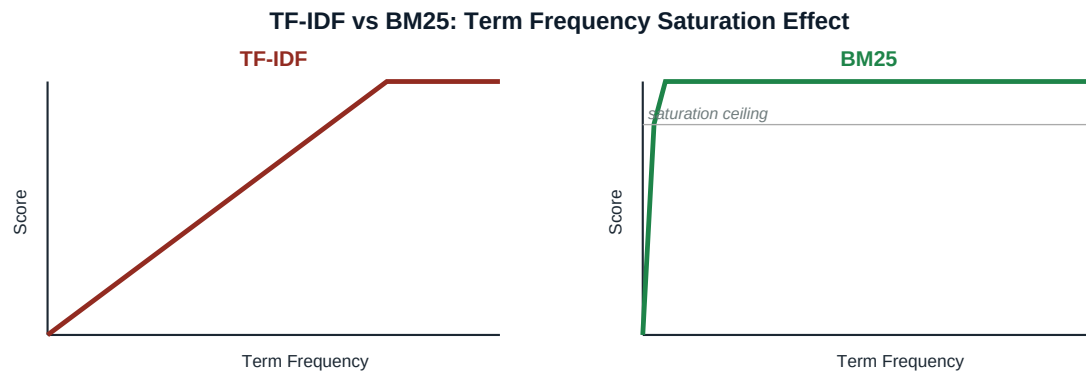


Figure 3 — IDF scores: common words like 'the' score near 0; rare domain-specific terms score near 1

2.2 TF Component — Term Frequency with Saturation

Unlike raw TF (which grows linearly), BM25's TF component saturates — a word appearing 100 times is not scored 100× higher than appearing 10 times. This prevents a single repeated term from dominating the score.

$f(q_i, D)$ = raw term count of q_i in



BM25 caps the influence of repeated terms — a word appearing 100× is not 100× more relevant than appearing 10×

Figure 4 — BM25 caps TF contribution; TF-IDF grows linearly with term frequency (problematic)

2.3 Length Normalisation

The term $|D|/avgdl$ (document length divided by average document length) normalises for document length. Without this, longer documents would always score higher simply because they contain more words. Parameter **b** controls how aggressively this normalisation is applied ($b=0$ means no normalisation; $b=1$ means full normalisation).

3. Key Parameters: k1 and b

BM25 has two tunable hyperparameters that control the retrieval behaviour. The defaults ($k_1=1.5$, $b=0.75$) work well for most corpora, but for document chunk retrieval in RAG you may want to tune them:

Parameter	Default	Effect of Increasing	Effect of Decreasing	Recommendation
k1 (TF saturation)	1.5	More weight to high-TF terms; less saturation	Faster saturation; term freq matters less	1.2–2.0 for most tasks; 1.5 is a solid default
b (length norm)	0.75	Penalises long docs more; favours short docs	Less length penalty; all docs treated equally	0.75 standard; lower (0.3) if chunks are uniform length
epsilon (IDF floor)	0.25	Higher floor for negative IDF terms; less aggressive	More aggressive IDF penalty for common terms	Keep default unless you have many common terms

Table 1 — BM25 hyperparameters and their effects

■ **Note** For RAG with fixed-size chunks (e.g. `chunk_size=200` as in your setup), all chunks are roughly the same length. In this case, length normalisation matters less — you can set $b=0.5$ or even $b=0$ to reduce its influence.

3.1 Example: Effect of k1 on Score

```
from rank_bm25 import BM25Okapi

corpus = ["the quick brown fox", "the fox jumped", "the lazy dog fox fox fox"]
tokenised = [doc.lower().split() for doc in corpus]

query = ["fox"]

# Default k1=1.5
bm25_default = BM25Okapi(tokenised, k1=1.5, b=0.75)
print("k1=1.5:", bm25_default.get_scores(query))
# >> [0.24, 0.31, 0.31] – doc 3 does NOT get 3x score despite 3x 'fox'

# Higher k1 – less saturation
bm25_high_k1 = BM25Okapi(tokenised, k1=3.0, b=0.75)
print("k1=3.0:", bm25_high_k1.get_scores(query))
# >> [0.21, 0.30, 0.45] – more spread, repeated 'fox' matters more
```

4. BM25 Variants in rank_bm25

The `rank_bm25` library ships four BM25 variants. For most RAG applications, `BM25Okapi` is the right choice:

Variant	Class in <code>rank_bm25</code>	Key Difference	Best For
BM25 Okapi (Standard)	<code>BM25Okapi</code>	Standard BM25 formula $k_1=1.5$, $b=0.75$ defaults	General-purpose — use this for most RAG applications
BM25L	<code>BM25L</code>	Applies a lower bound to TF to prevent negative IDF issues with very common terms	When your corpus has many very short documents
BM25Plus	<code>BM25Plus</code>	Adds a constant delta to TF so all terms get some credit; no zero-scoring for short docs	Short queries or corpora where length varies widely

Variant	Class in rank_bm25	Key Difference	Best For
BM25-Adpt (Adaptive)	BM25Adpt	Adapts k1 per term based on top-retrieved docs; more complex, slower to build	High-precision search where term saturation varies a lot

Table 2 — BM25 variants available in rank_bm25

```
from rank_bm25 import BM25Okapi, BM25L, BM25Plus

tokenised_corpus = [doc.lower().split() for doc in chunks]

bm25      = BM25Okapi(tokenised_corpus) # standard – use this first
bm25_l    = BM25L(tokenised_corpus)     # better for short documents
bm25_plus = BM25Plus(tokenised_corpus)  # no zero-scoring for any term
```

5. BM25 vs Vector Search — When to Use Which

Understanding when each retrieval method excels is critical for designing your RAG pipeline:

Dimension	BM25 (Sparse)	Vector Search (Dense)
Representation	Bag-of-words / token overlap	Dense embedding (1536 dims)
Strengths	Exact keyword match Product codes, names, acronyms Fast & no GPU needed No training required	Semantic similarity Paraphrases & synonyms Cross-language retrieval Context-aware ranking
Weaknesses	No semantic understanding Synonym blindness 'Car' ≠ 'automobile'	Misses exact terms Expensive (embeddings) Black-box scores
Index size	Small (token counts only)	Large (float vectors per chunk)
Query latency	Very fast (< 5ms typical)	Moderate (ANN search)
Setup complexity	pip install rank-bm25 3 lines of code	Embed all docs + Pinecone setup
Best queries	"BM25 k1 parameter" "invoice number INV-2024-007"	"How does BM25 handle term saturation?"
Ideal usage	Standalone OR hybrid first stage	Standalone OR hybrid reranking

Table 3 — BM25 (sparse) vs vector search (dense): strengths, weaknesses, and ideal use cases

★ **Key Concept** The best RAG systems use BOTH. BM25 for keyword precision, vector search for semantic understanding. Combined via RRF — see Section 11.

6. Installation: rank_bm25

The rank_bm25 library is a pure-Python implementation with no heavy dependencies — it only requires numpy.

```
# Install from PyPI
pip install rank-bm25

# Verify installation
python -c "from rank_bm25 import BM25Okapi; print('rank_bm25 ready')"
# >> rank_bm25 ready

# If using a requirements.txt
echo "rank-bm25==0.7.2" >> requirements.txt
pip install -r requirements.txt
```

■ **Note** Note the package name on PyPI is rank-bm25 (with a hyphen) but the import is rank_bm25 (with an underscore). This is a common source of confusion.

7. Building a BM25 Index over Document Chunks

BM25 requires a tokenised corpus — a list of lists where each inner list is the list of tokens for one chunk. The key steps are: load chunks, tokenise, and pass to BM25Okapi.

7.1 Simple tokenisation (lowercase split)

```
from rank_bm25 import BM25Okapi

# Assume `chunks` is a list of strings (your document chunk texts)
chunks = [
    "BM25 is a ranking function used in information retrieval.",
    "The k1 parameter controls term frequency saturation in BM25.",
    "Pinecone is a managed vector database for similarity search.",
    "Hybrid retrieval combines BM25 with dense vector embeddings.",
]

# Step 1: Tokenise – lowercase and split on whitespace
tokenised_corpus = [chunk.lower().split() for chunk in chunks]
# [['bm25', 'is', 'a', 'ranking', ...],
#  ['the', 'k1', 'parameter', 'controls', ...], ...]

# Step 2: Build the BM25 index
bm25 = BM25Okapi(tokenised_corpus)
# BM25Okapi automatically computes IDF for all terms across the corpus

print(f"Index built over {len(chunks)} chunks")
print(f"Unique terms in vocabulary: {len(bm25.idf)}")
```

7.2 Better tokenisation with stopwords removal

```
import re
from rank_bm25 import BM25Okapi

# Simple stopwords list (use NLTK or spaCy for production)
STOPWORDS = {
    "a", "an", "the", "is", "in", "it", "of", "and", "or", "to",
    "for", "with", "on", "at", "by", "this", "that", "was", "are",
}

def tokenise(text: str) -> list[str]:
    """Lowercase, remove punctuation, strip stopwords, min length 2."""
    text = text.lower()
    text = re.sub(r'[^a-z0-9\s]', ' ', text) # remove punctuation
    tokens = text.split()
    tokens = [t for t in tokens if t not in STOPWORDS and len(t) > 1]
    return tokens

# Tokenise all chunks
tokenised_corpus = [tokenise(chunk) for chunk in chunks]

# Build index
bm25 = BM25Okapi(tokenised_corpus)
print(tokenised_corpus[0])
# >> ['bm25', 'ranking', 'function', 'used', 'information', 'retrieval']
```

■ **Tip** Stopword removal significantly improves BM25 quality. Words like 'the', 'is', and 'a' appear in almost every document — removing them focuses BM25 on meaningful content words.

8. Querying the BM25 Index

`rank_bm25` provides two main query methods: `get_scores()` returns a score for every document, while `get_top_n()` returns the top-N highest-scoring documents. Both take the tokenised query (a list of tokens).

```
# Query must be tokenised the same way as the corpus
query = "BM25 retrieval ranking"
tokenised_query = tokenise(query)  # ['bm25', 'retrieval', 'ranking']

# Method 1: get_scores() – returns numpy array of scores for ALL chunks
scores = bm25.get_scores(tokenised_query)
print(scores)
# >> [1.42  0.87  0.00  0.95]  (score for each chunk in order)

# Method 2: get_top_n() – returns the top-N chunk texts directly
top_chunks = bm25.get_top_n(tokenised_query, chunks, n=3)
for i, chunk in enumerate(top_chunks):
    print(f"[{i+1}] {chunk}")

# Method 3: get_batch_scores() – score multiple queries at once
queries = [["bm25", "ranking"], ["vector", "embedding"]]
batch_scores = bm25.get_batch_scores(queries, list(range(len(chunks))))
```

8.1 Returning Scores with Indices

```
import numpy as np

def bm25_search(bm25, chunks, query: str, top_k: int = 5) -> list[dict]:
    """
    Search BM25 index and return top-k chunks with their scores and indices.
    Returns a list of dicts with 'chunk', 'score', 'index'.
    """
    tokenised_query = tokenise(query)
    scores = bm25.get_scores(tokenised_query)

    # Get top-k indices sorted by score (descending)
    top_indices = np.argsort(scores)[::-1][:top_k]

    results = []
    for idx in top_indices:
        if scores[idx] > 0:  # skip zero-scored chunks
            results.append({
                "index": int(idx),
                "chunk": chunks[idx],
                "score": float(scores[idx]),
            })
    return results

# Usage
results = bm25_search(bm25, chunks, "k1 parameter saturation", top_k=3)
for r in results:
    print(f"Score: {r['score']:.4f} | {r['chunk'][:80]}")
```

9. Full End-to-End Example with Your Document Setup

This example mirrors your existing RAG setup — loading PDFs with LangChain, splitting with RecursiveCharacterTextSplitter, and building a BM25 index over the resulting chunks (the same chunks you'd upsert to Pinecone):

[illegible]

```
# ■■■ 7. Test it ■■■■
```

```
results = bm25_retrieve("What are the four layers of the AI ecosystem?", top_k=5)
print(f"\nTop {len(results)} BM25 results:")
for r in results:
    print(f"      [{r['rank']}] score={r['score']:.4f} | "
          f"file={r['meta']['source_filename']} | "
          f"pg={r['meta']['page_number']}")
    print(f"           {r['text'][:120]}...")
```

■ **Note** The BM25 index lives entirely in memory (a Python object). It is NOT persisted to disk automatically. If your process restarts, you must rebuild it — see Section 10.3 for serialisation options.

10. Hybrid Retrieval: BM25 + Pinecone

In a hybrid RAG pipeline, you run BM25 and vector search in parallel for each query, then fuse the results. BM25 catches exact keyword matches; vector search catches semantic matches. Together they rarely miss relevant chunks.

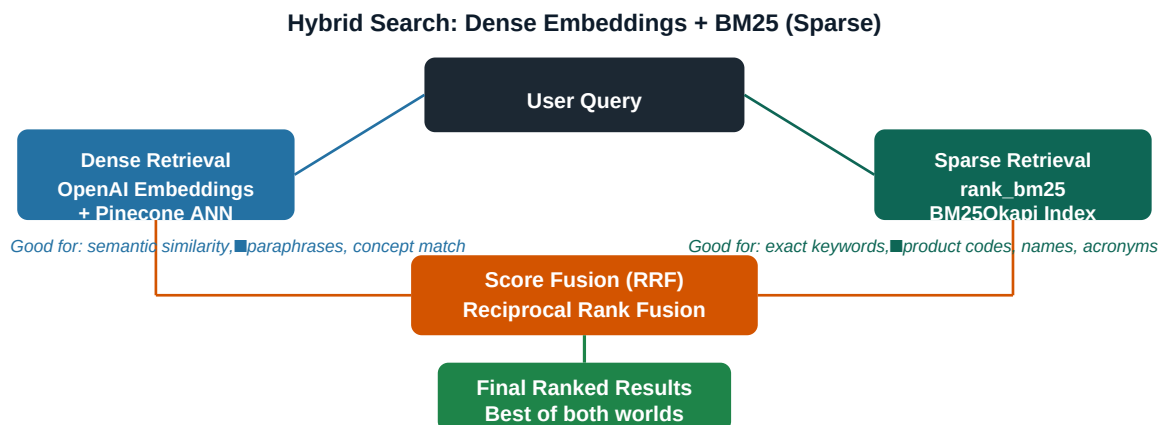


Figure 5 — Hybrid search architecture: two retrieval paths fused by Reciprocal Rank Fusion

[illegible]

```
# Sparse retrieval (BM25)
def sparse_retrieve(query: str, top_k: int = 10) -> list[dict]:
    scores = bm25.get_scores(tokenise(query))
    top_idx = np.argsort(scores)[::-1][:top_k]
    return [
        {"id": f"chunk-{idx}", "score": float(scores[idx]),
         "text": chunk_texts[idx], "source": "bm25"}
        for idx in top_idx if scores[idx] > 0
    ]
```

11. Reciprocal Rank Fusion (RRF)

Reciprocal Rank Fusion (RRF) is the standard way to merge results from two different retrieval systems. It works on **ranks** (positions), not raw scores, so it's robust even when BM25 and vector scores are on different scales.

[illegible]

```
for i, r in enumerate(results, 1):
    print(f" [{i}] RRF={r['rrf_score']:.4f} | src={r['source']} | {r['text'][:100]}")
```

■ **Tip** $k=60$ in RRF is a well-tested default. A higher k makes the fusion less sensitive to rank differences; a lower k amplifies the gap between top and lower-ranked results. In most RAG applications $k=60$ works well.

11.1 Serialising the BM25 Index (Persistence)

```
import pickle

# Save the BM25 index to disk
with open("bm25_index.pkl", "wb") as f:
    pickle.dump({
        "bm25": bm25,
        "chunk_texts": chunk_texts,
        "chunk metas": chunk_metas,
    }, f)
print("BM25 index saved to bm25_index.pkl")

# Load it back
with open("bm25_index.pkl", "rb") as f:
    saved = pickle.load(f)
bm25 = saved["bm25"]
chunk_texts = saved["chunk_texts"]
chunk_metas = saved["chunk metas"]
print(f"BM25 index loaded: {len(chunk_texts)} chunks")
```

12. Pros, Cons & Limitations of BM25

✓ Pros

- No training required — works out of the box on any corpus.
- Extremely fast at query time — pure Python on CPU, sub-millisecond for small corpora.
- No GPU, no embeddings, no API calls — zero inference cost.
- Excellent at exact keyword matching: product codes, IDs, proper nouns, acronyms.
- Transparent and interpretable — you can see exactly why a chunk scored high.
- Strong baseline: BM25 alone often beats naive vector search on keyword-heavy queries.
- Works well in hybrid mode: adds complementary signal to dense retrieval.
- Handles out-of-vocabulary (OOV) terms naturally — new terms just get zero IDF.

✗ Cons & Limitations

- No semantic understanding — 'car' and 'automobile' are completely different terms.
- No synonym handling — a query for 'physician' won't match 'doctor' without stemming.
- Word order ignored — 'dog bites man' and 'man bites dog' score identically.
- In-memory only — for 1M+ chunks, memory footprint becomes significant.
- Not persisted automatically — index must be rebuilt or pickled on each restart.
- Sensitive to tokenisation quality — garbage in, garbage out.
- Requires same tokenisation at index time and query time — easy to get wrong.
- Language-specific — stopwords and stemming need language-specific tuning.

12.1 Common Pitfalls

Pitfall	Fix
Tokenising query differently from corpus	Use the same tokenise() function for both corpus and query — always.
No stopword removal — 'the', 'is' dominate scores	Apply stopword removal before building index. Use NLTK's stopwords list.
Rebuilding BM25 on every request	Build once at startup; keep in memory. Pickle to disk for restarts.
Using BM25 score for cross-system comparison with vector scores	BM25 scores are not comparable to cosine similarity. Use RRF for fusion.
Forgetting that score=0 means no query term appears	Filter out zero-score results before returning to the user.

13. References

- [1] Robertson, S. & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval.
- [2] rank_bm25 — Python BM25 Library https://github.com/dorianbrown/rank_bm25
- [3] rank_bm25 PyPI page <https://pypi.org/project/rank-bm25/>
- [4] Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods. ACM SIGIR Conference on Research and Development in Information Retrieval.
- [5] Pinecone — Hybrid Search (BM25 + Dense) <https://docs.pinecone.io/guides/data/understanding-hybrid-search>
- [6] LangChain — RecursiveCharacterTextSplitter https://python.langchain.com/docs/how_to/recursive_text_splitter/
- [7] NLTK — Stopwords Corpus <https://www.nltk.org/book/ch02.html>
- [8] Elasticsearch BM25 documentation (reference implementation)
<https://www.elastic.co/guide/en/elasticsearch/reference/current/similarity.html>
- [9] Karpukhin et al. (2020). Dense Passage Retrieval for Open-Domain QA. EMNLP 2020. (Context for why hybrid outperforms either alone.)

Created on July 04, 2026 at 22:42. Library: rank_bm25 v0.7.2. Python 3.12. Companion to: Pinecone vector search, RecursiveCharacterTextSplitter, LangChain RAG pipeline.